UNIVERSIDAD DEL NORTE

Facultad de Ingeniería

Ingeniería en Informática

Asignatura: Programación IV

Docente: Prof. Marcos Arrieta Moreira

AVANCE 3

Proyecto Integrador: Menina Coiffure (Villa Elisa, Central, Paraguay)

Grupo N° 7

- Giovanni Rojas
- Braihan Vega
- Noelia Servían
- Danna Sanabria
- Thiago Rodriguez
- Lujan Ramírez

Asunción, Paraguay

Fecha de entrega: 30/10/2026

TABLA DE CONTENIDO

1. Introducción y Objetivos 1
    1.1. Contexto empresarial 1
2. Metodología Ágil: Sprints 3 y 4 2
    2.1. Tablero y criterio de actualización 2
    2.2. Sprint 3 (26/09 – 16/10) 2
    2.3. Sprint 4 (17/10 – 30/10) 3
    2.4. Cambios de alcance 4
    2.5. Flujo Kanban Acumulado 4
    2.6. Métricas y cuellos de botella 5
    2.7. Evidencia de ceremonias ágiles 6
3. Arquitectura de la Solución 7
    3.1. Tecnologías 7
    3.2. Servicios en ejecución 7
    3.3. Funcionalidades del sistema 7
4. Reglas de Negocio de las Reservas 9
5. Sesiones Centralizadas 10
6. Balanceo de Carga con HAProxy 11
    6.1. Integración en Docker Compose 11
    6.2. Archivo haproxy.cfg 12
    6.3. Algoritmo de balanceo y health checks 13
    6.4. Panel de estadísticas 14
    6.5. Página para ver el balanceo en vivo 14
7. Arquitecturas A y B 14
    7.1. Qué tiene cada arquitectura 15
    7.2. Por qué cada nodo tiene su propio PHP-FPM 15
    7.3. Diagrama del despliegue 15
8. Demostración de Failover 16
    8.1. Resultados 16
9. Evidencia de Pruebas 18
    9.1. Pruebas automatizadas 18
    9.2. Pruebas funcionales del frontend 18
10. Conclusiones y Próximos Pasos 19
11. Referencias 20

1. Introducción y Objetivos

1.1. Contexto empresarial

Menina Coiffure es una peluquería y centro de estética familiar de Villa Elisa, en el departamento Central. La fundó la Sra. Mónica en 2018 y hoy trabajan ahí tres personas: ella, que además de atender es la que maneja la agenda, una estilista y una asistente. Atienden de lunes a viernes de 8:00 a 17:00, en el local y también a domicilio. Cuando empezamos a investigar el negocio vimos que todas las citas se anotaban en cuadernos o quedaban en chats de WhatsApp. Por eso a veces se pisaban dos turnos, se daban citas fuera del horario y no quedaba registrado cuánto se le cobró realmente a cada clienta.

Para resolver eso armamos una aplicación web con Laravel 13 y PostgreSQL 18, con la interfaz hecha en Vue 3 e Inertia. Todo corre en contenedores con Docker Compose y lo administramos con Portainer. Desde el Avance 2 el sistema creció bastante: ahora tiene el panel completo para el equipo, una página pública del salón, cuentas para que las clientas pidan citas, permisos que la dueña puede activar para cada profesional y el inicio de sesión con KeyCloak.

2. Metodología Ágil: Sprints 3 y 4

2.1. Tablero y criterio de actualización

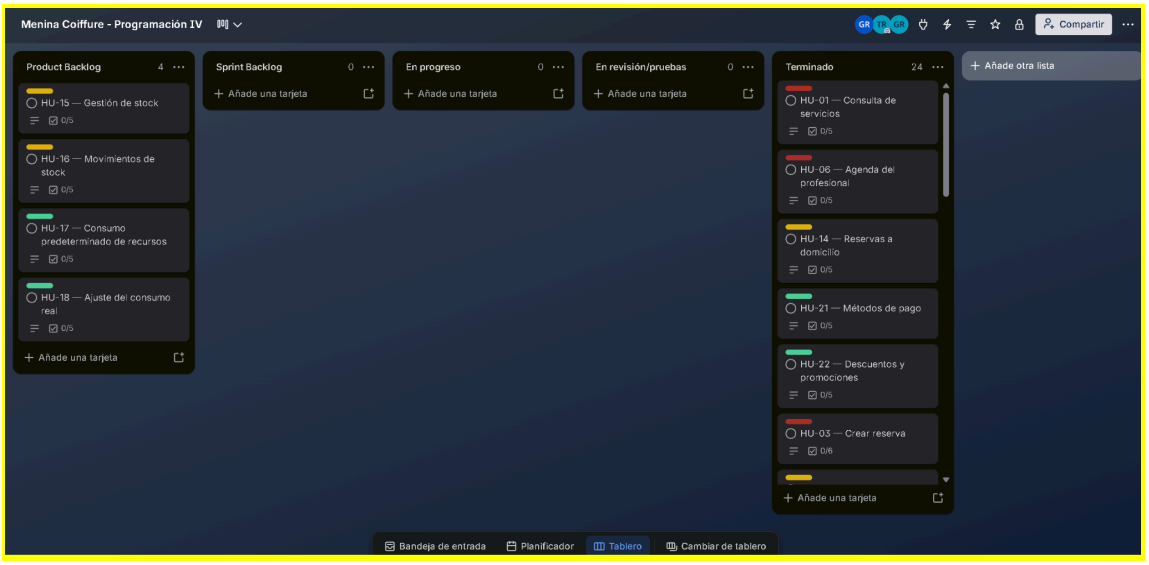
Seguimos trabajando con Scrum, y el tablero lo llevamos en Trello con vista Kanban. Las columnas son Product Backlog, Sprint Backlog, En progreso, En revisión / pruebas y Terminado. Los sprints duran entre 2 y 3 semanas, como pide la consigna:

Sprint	Período	Duración	Objetivo
Sprint 3	26/09/2026 – 16/10/2026	3 semanas	Frontend completo del panel y API completa con roles
Sprint 4	17/10/2026 – 30/10/2026	2 semanas	Clientas, configuración, KeyCloak, accesibilidad, HAProxy y failover

Para poner las fechas de cada tarjeta de los Sprints 3 y 4 usamos el historial de Git del repositorio, porque ahí queda registrado cuándo se hizo cada cosa y cualquiera lo puede comprobar. Usamos el mismo criterio para todas las tarjetas:

- **En progreso:** el día del primer commit de la tarea.
- **En revisión / pruebas:** el día del último commit, cuando ya estaba lista para probar.
- **Terminado:** el día en que se unió a main (con merge o Pull Request).

El problema es que Trello guarda la fecha real en la que se mueve una tarjeta y no deja cambiarla. Por eso anotamos las fechas del sprint en la descripción de cada tarjeta, junto con el commit que las respalda.



2.2. Sprint 3 (26/09 – 16/10)

En este sprint entraron las dos historias que quedaron abiertas del Sprint 2: HU-03, que estaba en revisión, y HU-06, que estaba en progreso. El objetivo principal fue armar la interfaz del panel y terminar la API.

Código	Tarea	Responsable	Estado al 16/10
HU-03	Crear reserva (viene del Sprint 2)	Giovanni · Thiago	Terminado
HU-06	Agenda del profesional (viene del Sprint 2)	Thiago · Giovanni	Terminado
T-04	Frontend base con Vue 3, Inertia y Tailwind	Thiago	Terminado
T-05	Interfaz base: menú, navegación, login y rutas protegidas	Thiago	Terminado
T-06	Sistema de diseño: paleta, tipografía y muestrario	Thiago	Terminado
HU-09 · HU-10 · HU-11	Servicios, categorías y cambio de precio con historial	Thiago	Terminado
HU-08	Turnos del personal con editor semanal	Thiago	Terminado
HU-23 · HU-24	Reportes de reservas, servicios e ingresos	Giovanni	Terminado
T-07	API completa, roles por usuario y rendimiento (88 pruebas)	Giovanni	Terminado
HU-05 · HU-13	Clientes: ficha, preferencias e historial de visitas	Giovanni (API) · Thiago (pantalla)	En progreso
HU-19 · HU-20	Pagos con seña, pago final y saldo	Giovanni (API) · Thiago (pantalla)	En progreso
HU-21 · HU-22	Métodos de pago y promociones	Giovanni (API) · Thiago (pantalla)	En progreso
HU-04	Modificar o cancelar una reserva	Giovanni · Thiago	En progreso
HU-14	Reservas a domicilio con traslado	Giovanni	En progreso

Resultado: terminamos 9 de las 14 tarjetas (64,3 %). Las otras 5 ya tenían la API lista el 16/10, pero las pantallas se hicieron en los días siguientes, así que pasaron al Sprint 4 como En progreso.

2.3. Sprint 4 (17/10 – 30/10)

En esta tabla están las 5 tarjetas que vinieron del Sprint 3 y las que se sumaron durante el sprint. Cada fecha corresponde a una columna del tablero.

Código	Tarea	Responsable	En progreso	En revisión	Terminado
HU-05 · HU-13	Clientes: ficha, preferencias e historial de visitas	Giovanni (API) · Thiago (pantalla)	16/10	18/10	22/10
HU-19 · HU-20	Pagos con seña, pago final y saldo	Giovanni (API) · Thiago (pantalla)	16/10	18/10	22/10
HU-21 · HU-22	Métodos de pago y promociones	Giovanni (API) · Thiago (pantalla)	16/10	18/10	22/10
HU-04	Modificar o cancelar una reserva	Giovanni · Thiago	16/10	22/10	22/10
HU-14	Reservas a domicilio con traslado	Giovanni	16/10	22/10	22/10
HU-07	Especialidades de cada profesional (ampliación)	Thiago	18/10	18/10	22/10
T-08	Nginx como proxy inverso	Braihan	21/10	21/10	22/10
T-09	Horario del salón y permisos del equipo	Giovanni	22/10	22/10	22/10
T-10	Página pública del salón	Giovanni	22/10	22/10	22/10
T-11	Pulido, buscador Ctrl+K, carga bajo demanda y accesibilidad	Thiago	23/10	23/10	24/10
T-12	Login con KeyCloak (OIDC y PKCE)	Giovanni	23/10	23/10	24/10
HU-26	Cuentas de clientas y pedidos de cita con aviso (nueva)	Giovanni	24/10	24/10	25/10
T-13	Diseño responsive en celular	Giovanni	26/10	26/10	26/10
T-14	HAProxy y arquitecturas A y B activas	Noelia · Giovanni	29/10	30/10	—
T-15	Demostración de failover	Giovanni	30/10	30/10	—
T-16	Documento del Avance 3	Thiago	28/10	—	—

Al 28/10 teníamos 13 de 16 tarjetas terminadas (81,3 %), una en progreso (este documento) y dos todavía en Sprint Backlog: HAProxy con las arquitecturas A y B, y la demostración de failover. Esas dos se trabajaron el 29 y el 30/10 y al cierre del sprint quedaron en revisión: ya funcionan y están probadas, solo falta unir la rama noeProxy a main.

2.4. Cambios de alcance

Durante el Sprint 4, después de hablar con la dueña, cambiamos algunas cosas del alcance. Esto ya está reflejado en el Product Backlog:

- **HU-15, HU-16, HU-17 y HU-18 (inventario de insumos):** las descartamos el 22/10. Por ahora el salón no necesita controlar el stock, así que sacamos el módulo del sistema (commit f90a598).
- **HU-08 (turnos del personal):** el 22/10 la cambiamos por un horario único del salón (T-09). Las profesionales atienden según la demanda y no tienen turnos fijos, entonces no tenía sentido cargarles turnos.
- **HU-26 (nueva):** cuentas de clientas y pedidos de cita. La clienta ve sus citas y pide nuevas desde su cuenta; la dueña las acepta o rechaza desde una bandeja y le llega un aviso por Telegram. En el Avance 1 esto estaba fuera del alcance, así que lo actualizamos en este avance.

2.5. Flujo Kanban Acumulado

Las tablas muestran cuántas tarjetas había en cada columna al final de cada día (SB = Sprint Backlog, EP = En progreso, ER = En revisión, T = Terminado). Los fines de semana solo aparecen si ese día hubo algún movimiento.

Sprint 3	SB	EP	ER	T		Sprint 4	SB	EP	ER	T
28/09	11	1	2	0		18/10	10	2	4	0
29/09	11	1	2	0		19/10	10	2	4	0
30/09	11	1	2	0		20/10	10	2	4	0
01/10	11	1	1	1		21/10	9	2	5	0
02/10	11	1	1	1		22/10	7	0	0	9
05/10	11	1	1	1		23/10	5	0	2	9
06/10	11	1	1	1		24/10	4	0	1	11
07/10	11	1	1	1		25/10	4	0	0	12
08/10	11	1	1	1		26/10	3	0	0	13
09/10	11	1	1	1		27/10	3	0	0	13
10/10	8	3	2	1		28/10	2	1	0	13
11/10	7	0	6	1						
12/10	7	0	6	1						
13/10	7	0	6	1						
14/10	7	0	6	1						
15/10	7	0	6	1						
16/10	0	5	0	9						

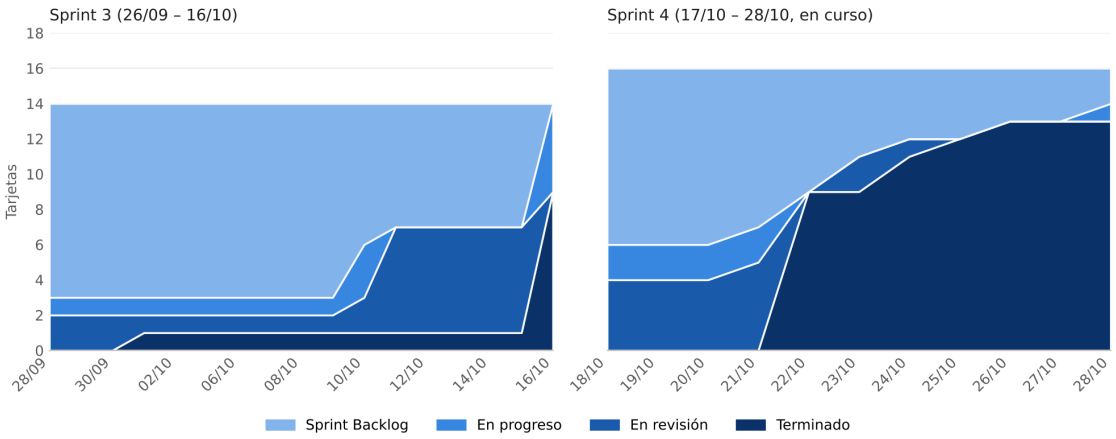


Figura 1. Diagrama de Flujo Acumulado de los Sprints 3 y 4 (datos al 28/10/2026).

2.6. Métricas y cuellos de botella

Métrica	Sprint 3	Sprint 4 (al 28/10)
Tarjetas terminadas	9 de 14	13 de 16
Máximo simultáneo en En revisión	6 (11/10 al 15/10)	5 (21/10)
Cycle Time promedio (En progreso → Terminado)	8,0 días	2,9 días

En el Avance 2 habíamos propuesto un WIP Limit de 2 tarjetas en En revisión / pruebas, pero no lo cumplimos en ninguno de los dos sprints. En el Sprint 3 se juntaron 6 tarjetas entre el 11/10 y el 15/10, y en el Sprint 4 llegaron a 5 el 21/10. En los dos casos pasó lo mismo: subíamos el trabajo a main por tandas (los merges del 16/10 y del 22/10) y no tarea por tarea, entonces las tarjetas ya terminadas se quedaban esperando juntas.

El Cycle Time bajó de 8,0 a 2,9 días. El del Sprint 3 salió alto por HU-03 y HU-06, que venían arrastradas desde el 23/09 (23 días cada una); si no las contamos, el promedio del Sprint 3 queda en 3,7 días. Las tarjetas con 0 días son tareas que se hicieron en un solo commit, y en esos casos Git no nos dice cuándo se empezaron de verdad.

Qué cambiamos: desde el 22/10 cada tarjeta se sube con su propio Pull Request apenas pasa la revisión (Pull Requests #1 a #10).

2.7. Evidencia de ceremonias ágiles

Estas son las ceremonias del Sprint 4.

Ceremonia	Fecha	Modalidad	Asistentes
Sprint Planning (Sprint 4)	05-11-26	Asíncrona por WhatsApp	Equipo
Daily Standups	09-11-26	Asíncrona por WhatsApp	Equipo

3. Arquitectura de la Solución

3.1. Tecnologías

Capa	Tecnología
Backend	Laravel 13 sobre PHP 8.4 (FPM)
Base de datos	PostgreSQL 18
Frontend	Vue 3.5, Inertia 3, Tailwind CSS 4 y Vite 8
Identidad	KeyCloak 26 (OpenID Connect con PKCE), opcional junto al inicio de sesión propio
Servidor web	nginx (alpine), dos instancias: una por nodo, cada una con su propio PHP-FPM
Contenedores	Docker Compose y Portainer CE
Balanceador	HAProxy 3.0 (imagen haproxy:3.0-alpine)
Pruebas y API	PHPUnit, OpenAPI con Swagger UI

3.2. Servicios en ejecución

Todos los servicios están en la misma red privada de Docker, menina_net:

Servicio	Contenedor	Imagen	Puerto publicado	Función
haproxy	menina_haproxy	haproxy:3.0-alpine	8000 / 8404	Entrada única: reparte el tráfico entre A y B y muestra el panel de estadísticas
nginx	menina_nginx	nginx:alpine	— (interno 80)	Nodo A: servidor web, pasa el PHP a app:9000
app	menina_app	php:8.4-fpm (propia)	—	Nodo A: lógica de negocio y API
nginx_b	menina_nginx_b	nginx:alpine	— (interno 80)	Nodo B: servidor web, pasa el PHP a app_b:9000
app_b	menina_app_b	php:8.4-fpm (propia)	—	Nodo B: la misma aplicación que app
db	menina_db	postgres:18	5432	Base de datos, sesiones, caché y colas
keycloak	menina_keycloak	keycloak:26.0	8080	Inicio de sesión, registro y recuperación de contraseña
portainer	menina_portainer	portainer-ce	9443 / 9001	Administración del stack

En el Avance 2 la imagen usaba PHP 8.4.26. El 24/10 la reconstruimos con `docker compose build --pull app` para pasar a PHP 8.4.27, que corrige las vulnerabilidades CVE-2026-109618 y CVE-2026-109619 (PHP2Shell), publicadas el 21/10.

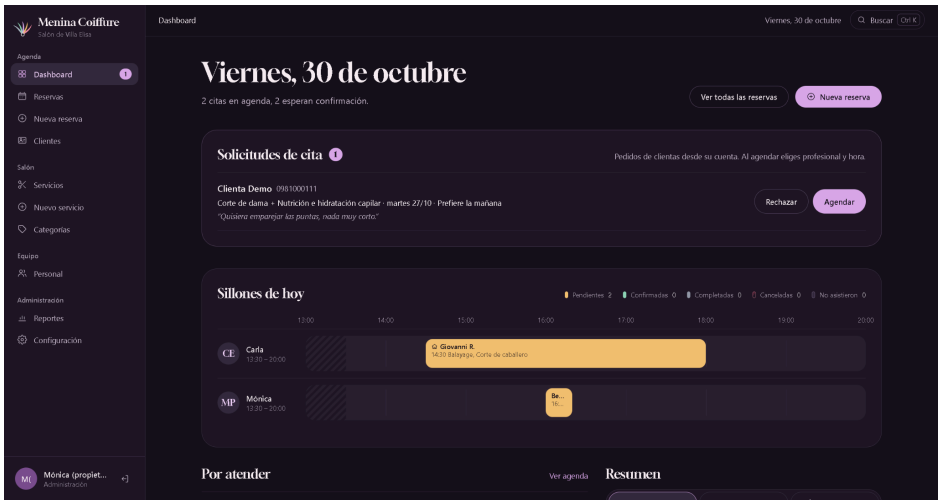
3.3. Funcionalidades del sistema

- **Panel del equipo:** en el Dashboard se ven las citas del día y los pedidos que mandan las clientas. También tiene la agenda de reservas con filtros, la carga de

reservas en el salón o a domicilio, cambios y cancelaciones, pagos con seña y saldo, fichas de clientas con preferencias e historial, servicios con su historial de precios, personal con especialidades, reportes y una pantalla de configuración (horario del salón, días sin atención, permisos, métodos de pago, promociones por día y especialidades).

- **Página pública del salón (/salon):** muestra el catálogo, las promociones del día, el horario y un botón para escribir por WhatsApp.
- **Cuenta de la clienta (/mi-cuenta):** la clienta ve sus citas y puede pedir una nueva eligiendo el día, la hora, los servicios y dónde quiere que la atiendan.
- **Roles y permisos:** la administración puede hacer todo. Cada profesional ve su propia agenda, y la dueña le puede habilitar hasta cuatro permisos más a cada una.
- **Accesibilidad y rendimiento:** revisamos las pantallas del panel con axe-core y bajamos los problemas de 94 a 0. También hicimos que cada pantalla se cargue recién cuando se abre, y con eso el archivo inicial pasó de 527 kB a 193 kB.





4. Reglas de Negocio de las Reservas

Toda la lógica de las reservas está en `src/app/Services/ReservaService.php` y corre dentro de `DB::transaction`. Eso quiere decir que una reserva se guarda entera o no se guarda nada. Las reglas que aplicamos son:

#	Regla	Descripción
1	Sin solapamiento	Una profesional no puede tener dos citas activas superpuestas. En las citas a domicilio se considera también el tiempo de traslado.
2	Horario del salón	La cita debe caber en el horario del salón, salvo que se marque como fuera de horario (06:00 a 22:00) por alguien con ese permiso.
3	Servicios activos	No se puede reservar ni agregar un servicio inactivo.
4	Precio congelado	Cada línea guarda el precio vigente a la fecha de la cita; los cambios posteriores del catálogo no la alteran.
5	Estados finales	Una reserva completada o cancelada no puede modificarse ni reactivarse. Las pendientes y confirmadas pueden editarse.

Concurrencia entre nodos. Ahora que hay dos nodos, puede pasar que dos personas reserven a la misma profesional en el mismo momento y que cada pedido caiga en un nodo distinto. Para que no se guarden las dos citas, al crear una reserva el sistema bloquea el registro de la profesional hasta que termina la transacción. Así el segundo pedido espera al primero y, cuando le toca, ya encuentra el solapamiento y se rechaza por la Regla 1.

5. Sesiones Centralizadas

Con un solo servidor, Laravel guarda las sesiones en archivos dentro de ese mismo servidor. Con dos nodos eso trae un problema: alguien inicia sesión en el nodo A, la siguiente petición le cae al nodo B, que no tiene ese archivo, y lo saca de la sesión. Para que no pase, guardamos las sesiones, la caché y las colas en la base de datos, que es la misma para los dos nodos:

```
SESSION_DRIVER=database
SESSION_LIFETIME=120
CACHE_STORE=database
QUEUE_CONNECTION=database
```

Así cualquier nodo lee la misma información y a la persona le da igual cuál la atiende. Esto lo dejamos listo en el Avance 2 y fue lo que nos permitió poner las arquitecturas A y B detrás de HAProxy sin tocar el código de la aplicación. El 30/10 lo comprobamos: iniciamos sesión con KeyCloak y las peticiones siguientes fueron alternando entre A y B sin que se perdiera la sesión.

6. Balanceo de Carga con HAProxy

HAProxy está delante de las dos arquitecturas y decide a cuál mandar cada petición. La consigna pide tres cosas:

- **Algoritmo de balanceo:** `roundrobin` (por turnos) o `leastconn` (al nodo que tiene menos conexiones abiertas).
- **Health checks:** HAProxy le pregunta cada cierto tiempo a cada nodo si está bien y, si uno no contesta, deja de mandarle tráfico.
- **Panel de métricas:** HAProxy Stats en el puerto 8404.

La primera versión (29/10, commit d5a0a02) dejó a HAProxy como punto de entrada, pero tenía un solo servidor en el backend, así que en la práctica no balanceaba nada y no se podía hacer failover. Además el health check pedía la página principal (`GET /`), y como esa ruta abre una sesión, cada chequeo creaba una fila nueva en la tabla `sessions` cada 2 segundos. El 30/10 la rehicimos en la misma rama con los dos nodos y la probamos con Docker. Lo que sigue describe esa versión final.

6.1. Integración en Docker Compose

HAProxy es un servicio más dentro del `docker-compose.yml` y ahora es la única entrada al sistema. Publica el puerto 8000, que antes publicaba nginx, y los nginx de los dos nodos solo se ven dentro de la red menina_net. De esta forma ninguna petición llega a la aplicación sin pasar por el balanceador, y para el usuario la dirección sigue siendo la misma: http://localhost:8000.

Para tener el nodo B agregamos dos servicios: `app_b`, que es otro contenedor de PHP-FPM con el mismo código, y `nginx_b`, que le pasa las peticiones a `app_b`. Los dos nodos usan el mismo archivo `nginx.conf`. Para eso lo montamos como plantilla y cada nginx recibe en la variable `PHP_FPM_HOST` a qué PHP le tiene que mandar las peticiones (`app` o `app_b`). Así no hay dos archivos casi iguales que mantener. En el `nginx.conf` la línea quedó así: `fastcgi_pass ${PHP_FPM_HOST}:9000;`

Estos son los servicios nuevos del `docker-compose.yml` (sacamos `restart` y `networks`, que son iguales en todos). El nodo A es igual, pero con `PHP_FPM_HOST: app` y `hostname: nodo-a`:

```
app_b:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: menina_app_b
  hostname: nodo-b
  volumes:
    - ./src:/var/www
    - vendor:/var/www/vendor
  depends_on:
    db:
      condition: service_healthy

nginx_b:
  image: nginx:alpine
  container_name: menina_nginx_b
  expose:
    - "80"
  environment:
```

```
      PHP_FPM_HOST: app_b
    volumes:
      - ./src:/var/www
      - ./nginx.conf:/etc/nginx/templates/default.conf.template:ro
    depends_on:
      - app_b

  haproxy:
    image: haproxy:3.0-alpine
    container_name: menina_haproxy
    ports:
      - "8000:80"
      - "8404:8404"
    volumes:
      - ./haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg:ro
    depends_on:
      - nginx
      - nginx_b
```

6.2. Archivo haproxy.cfg

Este es el archivo completo. Le sacamos los comentarios para que entre mejor en la página, y la línea de `default-server` está cortada en dos solo acá; en el archivo real va en una sola línea.

```
global
    log stdout format raw local0
    maxconn 2000

resolvers docker
    nameserver dns 127.0.0.11:53
    resolve_retries 3
    timeout resolve 1s
    timeout retry   1s
    hold valid      10s

defaults
    log global
    mode http
    option httplog
    option dontlognull
    option forwardfor
    retries 2
    option redispatch
    timeout connect         1s
    timeout client          30s
    timeout server          30s
    timeout http-request    10s
    timeout http-keep-alive 10s
    timeout queue           30s
    timeout check           3s

frontend http_front
    bind *:80
    default_backend nodos_web

backend nodos_web
    balance leastconn
    option httpchk
    http-check send meth GET uri /up ver HTTP/1.1 hdr Host localhost
    http-check expect status 200
    http-response set-header X-Servido-Por %s
    default-server check inter 1s fall 2 rise 2 maxconn 30
```

```
                    resolvers docker init-addr libc,none
    server nodo_a nginx:80
    server nodo_b nginx_b:80

frontend stats_front
    bind *:8404
    stats enable
    stats uri /stats
    stats refresh 5s
    stats show-legends
    stats auth admin:admin
    stats admin if TRUE
```

Esto es lo que hace cada parte:

Directiva	Para qué sirve
`resolvers docker`	Usa el DNS interno de Docker para encontrar los nodos por su nombre. Gracias a esto HAProxy arranca aunque un nodo esté apagado y lo vuelve a encontrar si el contenedor se recrea con otra IP.
`option forwardfor`	Agrega la cabecera X-Forwarded-For con la IP real de quien visita. nginx la toma como IP real (set_real_ip_from 172.16.0.0/12), así en los registros aparece la IP del visitante y no la del balanceador.
`retries 2` y `option redispatch`	Si no se puede conectar con un nodo, HAProxy reintenta y manda la petición al otro. Esto es lo que hace que la caída de un nodo no le dé error al usuario.
`timeout connect 1s`	Tiempo máximo para conectar con un nodo. Dentro de Docker conectar tarda menos de 1 ms, así que 1 s alcanza y hace que el reintento al otro nodo sea rápido (ver sección 8.2).
`timeout client / server / http-request / queue`	Límites para que una conexión colgada o un cliente muy lento no ocupen recursos para siempre.
`balance leastconn`	Manda cada petición al nodo que tiene menos trabajo en ese momento (sección 6.3).
`option httpchk` y `http-check`	El health check pide /up, la ruta de salud que ya trae Laravel, y espera un código 200.
`http-response set-header X-Servido-Por %s`	Agrega a cada respuesta el nombre del nodo que la atendió (nodo_a o nodo_b). Lo usamos para mostrar el balanceo.
`default-server`	Configuración común de los dos servidores: chequeo cada 1 s, como mucho 30 peticiones a la vez por nodo y uso del DNS de Docker.
`server nodo_a` y `server nodo_b`	Los dos nodos: nginx (Arquitectura A) y nginx_b (Arquitectura B).
`frontend stats_front`	Panel de estadísticas en el puerto 8404, ruta /stats, con usuario y contraseña. Con `stats admin` se puede poner un nodo en mantenimiento desde el mismo panel.

6.3. Algoritmo de balanceo y health checks

Al principio se había elegido `roundrobin`, que reparte por turnos: una petición a A, la siguiente a B, y así. El problema es que en nuestro sistema no todas las peticiones

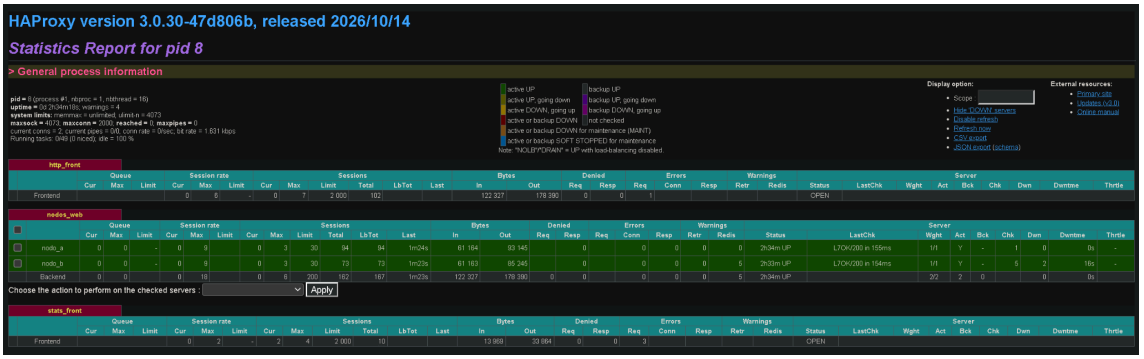
tardan lo mismo. Abrir la agenda es rápido, pero un reporte de ingresos o una reserva con varios servicios hace más consultas a la base y tarda más. Con roundrobin puede pasar que un nodo esté ocupado con algo pesado y aun así le siga tocando su turno. Por eso cambiamos a `leastconn`, que mira cuántas peticiones tiene abiertas cada nodo y manda la nueva al que está más libre. Cuando los dos están libres se turnan, así que con poco tráfico se comporta igual que roundrobin.

Para el health check usamos la ruta `/up`, que Laravel ya trae para esto. A diferencia de la página principal, `/up` no abre sesión, así que no llena la tabla `sessions`. Lo comprobamos: en 10 segundos de chequeos no apareció ninguna sesión nueva. HAProxy consulta cada nodo cada 1 segundo; si falla 2 veces seguidas lo marca como DOWN y deja de mandarle tráfico, y cuando responde bien 2 veces lo vuelve a poner en UP. O sea que una caída se detecta en unos 2 segundos.

También le pusimos un límite de 30 peticiones a la vez por nodo (`maxconn 30`). PHP-FPM viene configurado con 5 procesos por contenedor, entonces si llegan muchas peticiones juntas es mejor que esperen en la cola de HAProxy y no que se amontonen en un nodo hasta saturarlo.

6.4. Panel de estadísticas

El panel está en http://localhost:8404/stats y pide usuario y contraseña (admin / admin, porque es un entorno de desarrollo). Para cada nodo muestra si está UP o DOWN, el resultado del último chequeo, cuántas sesiones tiene abiertas y cuántas atendió en total, y los bytes que pasaron. Desde ahí también se puede poner un nodo en mantenimiento sin tocar Docker.



6.5. Página para ver el balanceo en vivo

El panel de HAProxy muestra números, pero para la exposición queríamos algo donde se vea a simple vista cómo se reparten las peticiones. Por eso hicimos una página aparte en http://localhost:8000/balanceo. Desde ahí se mandan peticiones reales que pasan por HAProxy, y la página va mostrando:

- cuántas peticiones atendió cada nodo y qué porcentaje del total representa;
- una tira de colores con las últimas 60 peticiones (violeta para A y ámbar para B);
- si cada nodo está atendiendo o dejó de responder;

- un registro con el nodo que eligió HAProxy, el PHP que respondió y cuánto tardó.

Tiene un botón para mandar 20 peticiones de golpe y otro para dejar tráfico continuo, y se puede simular que cada petición tarda más para que se note cómo trabaja leastconn. La página consulta `/api/balanceo/ping`, una ruta que devuelve el nombre del contenedor que atendió (nodo-a o nodo-b) y que no abre sesión. Para saber qué nodo eligió HAProxy, lee la cabecera X-Servido-Por de cada respuesta.



7. Arquitecturas A y B

En nuestro caso, las arquitecturas A y B son dos nodos que corren la misma aplicación, y HAProxy reparte el trabajo entre los dos para que ninguno se sobrecargue. Decidimos que sean iguales porque así cualquiera de los dos puede atender cualquier pedido, y si uno se cae el otro sigue solo sin que cambie nada para el usuario.

7.1. Qué tiene cada arquitectura

Componente	Arquitectura A	Arquitectura B
Nombre en HAProxy	nodo_a	nodo_b
Servidor web	nginx (menina_nginx)	nginx_b (menina_nginx_b)
PHP-FPM	app (menina_app)	app_b (menina_app_b)
Nombre del host	nodo-a	nodo-b
Código	carpeta src (la misma)	carpeta src (la misma)
Base de datos	menina_db (compartida)	menina_db (compartida)

Los dos nodos comparten el código (la carpeta `src` se monta en los dos), las dependencias de Composer (el volumen `vendor`) y la base de datos, y por lo tanto

también las sesiones y la caché. Lo único que cambia es a qué PHP-FPM le pasa las peticiones cada nginx.

7.2. Por qué cada nodo tiene su propio PHP-FPM

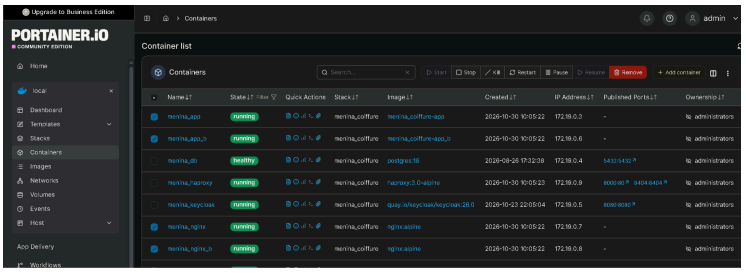
Una opción más simple era poner dos nginx que le pasen todo al mismo contenedor `app`. La descartamos porque en esta aplicación el trabajo pesado lo hace PHP, no nginx. Si los dos nginx compartieran un solo PHP-FPM, el cuello de botella seguiría siendo uno solo y balancear no serviría de mucho; además, si ese contenedor se caía, se caían los dos nodos a la vez.

#	Opción	Qué pasa
1	Dos nginx y un solo app	Usa menos contenedores, pero PHP sigue siendo uno solo: la carga no se reparte y si app se cae, se caen A y B.
2	Un nginx y un app por nodo (la que usamos)	La carga de PHP sí se reparte y cada nodo puede fallar sin arrastrar al otro. A cambio hay un contenedor de PHP más, que usa algo más de memoria.

7.3. Diagrama del despliegue

```
                Navegador
                    |
          http://localhost:8000
                    |
HAProxy  (leastconn + health check /up)
       |                        |
Arquitectura A           Arquitectura B
nginx -> app             nginx_b -> app_b
       |                        |
       +----> PostgreSQL <------+
       (datos, sesiones y caché)
```

Figura 2. Despliegue con HAProxy delante de las arquitecturas A y B.



8. Demostración de Failover

Con esta prueba queríamos ver que el sistema siga funcionando cuando una de las dos arquitecturas se cae. La hicimos el 30/10 y es la misma que vamos a mostrar en vivo. Los pasos son:

1. Con los dos nodos encendidos, abrimos el panel de HAProxy (puerto 8404) y la página de balanceo, y comprobamos que A y B están en **UP**.

2. Iniciamos sesión en la aplicación (pasando por HAProxy) y prendemos el tráfico continuo en la página de balanceo.
3. Apagamos la Arquitectura B con `docker compose stop nginx_b`.
4. En unos 2 segundos el panel muestra a nodo_b en **DOWN** y la página de balanceo lo marca como "Sin respuesta". Todo el tráfico pasa al nodo A.
5. Seguimos usando la aplicación (por ejemplo, abrimos la agenda). Responde desde A y la sesión sigue abierta, porque está guardada en la base.
6. Volvemos a encender B con `docker compose start nginx_b`. HAProxy lo detecta, lo pasa a **UP** y vuelve a repartir entre los dos.

8.1. Resultados

Estos son los datos de la prueba que hicimos el 30/10 con Docker, mandando peticiones de forma continua mientras apagábamos y volvíamos a prender la Arquitectura B:

Qué medimos	**Resultado**
Ráfaga de 20 peticiones al mismo tiempo (con los dos nodos UP)	10 las atendió A y 10 las atendió B
Peticiones durante la prueba de failover	39 (una cada 0,15 s aproximadamente)
Errores	0
Tiempo hasta que HAProxy marcó a B como DOWN	Unos 2 segundos
Petición más lenta mientras B se caía	1,2 segundos (la terminó atendiendo A)
Tiempo hasta que B volvió a recibir tráfico	Unos 4 segundos después de encenderlo
Sesión iniciada con KeyCloak	Se mantuvo: 6 de 6 consultas respondieron bien, alternando entre A y B

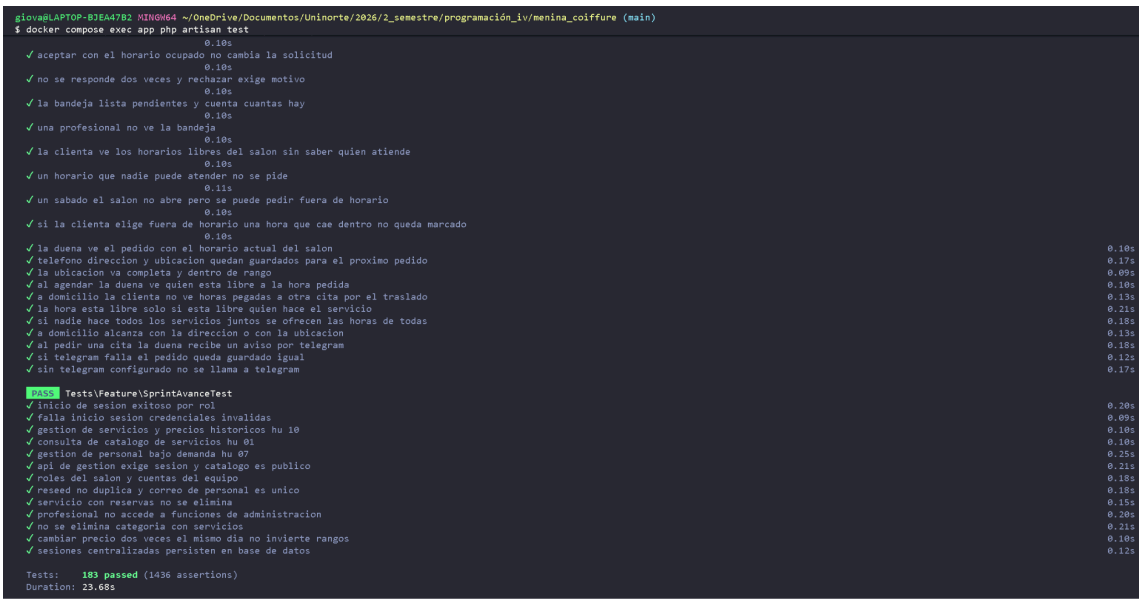
9. Evidencia de Pruebas

9.1. Pruebas automatizadas

La suite de PHPUnit pasó de 19 pruebas en el Avance 2 a 183 pruebas en 14 archivos:

Archivo	Pruebas	Cobertura
ReservaTest	51	Reglas de negocio, estados, modificación y cancelación
SolicitudCitaTest	28	Pedidos de cita de clientas y bandeja de la propietaria
KeycloakTest	16	Inicio de sesión por OIDC con PKCE y cierre de sesión
AgendaAjustesTest	14	Horarios ofrecidos, fuera de horario y cierres por rango
SprintAvanceTest	13	Criterios de aceptación de los avances anteriores
EspecialidadHorarioTest	12	Especialidades y horario del salón
PermisoTest	11	Permisos por profesional
ExpedienteTest	10	Ficha, preferencias e historial de clientas
PagoTest	9	Pagos con seña, pago final y saldo
ReporteTest	7	Reportes de reservas, servicios e ingresos
SalonPublicoTest	6	Página pública del salón
BalanceoTest	4	Página /balanceo y ruta de ping (que no abra sesión)
ExampleTest (Feature y Unit)	2	Pruebas base de Laravel

Corrimos `docker compose exec app php artisan test` con el sistema ya balanceado y pasaron las 183 pruebas (1.436 aserciones) en unos 22 segundos.



9.2. Pruebas funcionales del frontend

Con un navegador automatizado (Playwright) probamos la navegación entre pantallas, el buscador rápido y que cada pantalla se cargue recién cuando se abre: pasaron los 15 casos. La accesibilidad la revisamos con axe-core en 19 vistas y no quedaron problemas en las pantallas del panel. Las pruebas del balanceo con Docker están en la sección 8.

10. Conclusiones

Proceso. El tablero quedó al día con los Sprints 3 y 4, con fechas sacadas del historial de Git. Como integrábamos el trabajo por tandas, se juntaban tarjetas en revisión y nos pasamos del WIP Limit que habíamos propuesto. Desde que cada tarea va con su propio Pull Request, el Cycle Time bajó de 8,0 a 2,9 días.

Producto. El sistema empezó como un CRUD de reservas y ahora es una aplicación completa para el salón y sus clientas, con 183 pruebas automatizadas y las sesiones en la base de datos, listas para trabajar con varios nodos.

Infraestructura. HAProxy ya reparte el tráfico entre las arquitecturas A y B con leastconn, revisa cada nodo con health checks a /up y tiene el panel en el puerto 8404. En la prueba de failover apagamos un nodo con tráfico corriendo y no hubo ningún error.

11. Referencias

- Laravel. Documentación oficial. https://laravel.com/docs
- PostgreSQL Global Development Group. PostgreSQL Documentation. https://www.postgresql.org/docs
- HAProxy Technologies. HAProxy Configuration Manual. https://docs.haproxy.org
- Keycloak. Server Administration Guide. https://www.keycloak.org/documentation
- Vue.js. Guía oficial. https://vuejs.org/guide
- Inertia.js. Documentación. https://inertiajs.com
- Docker. Docker Compose documentation. https://docs.docker.com/compose/
- Portainer. Portainer Documentation. https://docs.portainer.io/
- W3C. Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/
- Schwaber, K. y Sutherland, J. (2020). The Scrum Guide. https://scrumguides.org
- Anderson, D. J. (2010). Kanban: Successful Evolutionary Change for Your Technology Business. Blue Hole Press.